

Documento de Fundamentação Técnica: Transição do Monólito aos Microsserviços

Projeto: Arquitetura de Soluções e DevOps – Startup AppFood

Este memorial descritivo tem por objetivo fundamentar as escolhas tecnológicas e arquiteturais para a reestruturação do ecossistema da startup AppFood. A migração da atual arquitetura monolítica — que centraliza Vendas, Pagamentos, Usuários e Pedidos em uma única base de código e banco de dados — para uma topologia de **Microsserviços Orquestrados** visa eliminar gargalos de concorrência, evitar falhas em cascata e garantir a escalabilidade horizontal e a resiliência do negócio.

Abaixo, detalha-se cada uma das tecnologias adotadas no novo ecossistema, descrevendo sua essência técnica e a justificativa exata de sua aplicação frente aos desafios críticos enfrentados pela AppFood.

Tecnologias Adotadas e Fundamentação

Git (Controle de Versão Distribuído)

O Git é um sistema de controle de versão distribuído projetado para rastrear alterações no código-fonte durante o desenvolvimento de software. Ele permite que múltiplos desenvolvedores trabalhem de forma concorrente no mesmo projeto, mantendo um histórico completo, ramificações (*branches*) isoladas para novas funcionalidades e mecanismos seguros de fusão (*merge*) de código.

Motivo da aplicação no cenário da AppFood:

No monólito atual, todas as equipes modificam o mesmo repositório centralizado, gerando conflitos frequentes e riscos operacionais. Com a adoção de microsserviços, o sistema será desacoplado em código isolado, exigindo que cada módulo (Vendas, Pagamentos, Usuários e Pedidos) possua seu próprio repositório Git dedicado. Isso garante total independência de desenvolvimento, permitindo que alterações ou correções em um módulo específico ocorram sem interferir no ciclo de vida dos demais.

Pipeline de CI/CD (Integração Contínua e Entrega Contínua)

Uma esteira de CI/CD é um conjunto de práticas e ferramentas automatizadas que

executam etapas padronizadas de build, análise estática de código, testes automatizados e empacotamento. O pipeline assegura que qualquer alteração de código passe por validações rigorosas antes de ser integrada e liberada para os ambientes de homologação ou produção, eliminando processos manuais propensos a falhas.

Motivo da aplicação no cenário da AppFood:

A AppFood sofria com instabilidades causadas por exceções não tratadas que derrubavam toda a aplicação. A implementação de uma esteira robusta — contendo etapas obrigatórias de *Build*, *Lint & Quality*, *Test*, *SAST* (Análise Estática de Segurança) e *Package* — garante que vulnerabilidades de segurança, falhas sintáticas e bugs em dependências sejam bloqueados automaticamente antes de chegarem ao ambiente produtivo, blindando o ecossistema contra impactos sistêmicos.

Docker (Containerização)

O Docker é uma tecnologia de virtualização baseada em containers que empacota uma aplicação junto com todas as suas dependências, bibliotecas e arquivos de configuração em uma imagem leve, isolada e imutável. Isso garante que a aplicação execute de forma idêntica e previsível em qualquer ambiente computacional, do notebook do desenvolvedor até o servidor de produção.

Motivo da aplicação no cenário da AppFood:

No monólito, todos os quatro domínios compartilhavam exatamente o mesmo ecossistema de tempo de execução e poder computacional. Com o Docker, cada microsserviço da AppFood será isolado em seu próprio container otimizado (utilizando imagens base seguras e enxutas como Alpine ou Distroless). Isso resolve o problema de disputa de recursos, permitindo o empacotamento estável e versionado de cada componente de forma independente.

2.4. Kubernetes (Orquestração de Containers)

O Kubernetes é uma plataforma de código aberto para automatizar a implantação, o dimensionamento (*scaling*) e a gestão de aplicações containerizadas em clusters de servidores. Ele gerencia o ciclo de vida dos containers, realiza balanceamento de carga interno, auto-recuperação (*self-healing*) em caso de falhas e distribuição inteligente de recursos de hardware.

Motivo da aplicação no cenário da AppFood:

O Kubernetes resolve os maiores gargalos operacionais da startup:

Isolamento e Resiliência contra Falhas: Através do padrão *Database-per-Service*, cada microsserviço terá seu próprio banco de dados isolado. Isso impede que a exaustão de conexões gerada por picos de acesso no módulo de *Pedidos* afete a autenticação de *Usuários*. Além disso, se o módulo de *Pagamentos* sofrer uma falha ou exceção não tratada, o Kubernetes conterà o erro e reiniciará a instância afetada automaticamente, evitando que a aplicação inteira caia.

Escalabilidade Dinâmica: Permite dimensionar o número de réplicas de Pods de acordo com a demanda em tempo real.

Prometheus (Coleta Distribuída de Métricas)

O Prometheus é um sistema de monitoramento e alerta de código aberto baseado em séries temporais. Ele opera ativamente utilizando um modelo *pull*, coletando métricas de performance (como consumo de CPU, memória, latência e taxas de erro HTTP) diretamente de pontos de extremidade (*endpoints*) expostos pelos serviços monitorados.

Motivo da aplicação no cenário da AppFood:

Em uma arquitetura descentralizada, o rastreamento manual de falhas torna-se inviável. O Prometheus será implementado para raspar métricas vitais de cada Pod no cluster Kubernetes. Através de expressões direcionadas (como a mensuração de picos de erros internos de transações de pagamento nos últimos 5 minutos via `http_requests_total`), a ferramenta fornece a base analítica necessária para detectar anomalias de performance e gargalos instantaneamente.

Grafana (Visualização Analítica e Dashboards)

O que é em sua essência técnica:

O Grafana é uma plataforma multi-plataforma de visualização analítica e criação de painéis interativos (*dashboards*). Ele se integra nativamente a bases de dados de séries temporais (como o Prometheus) para transformar métricas brutas em gráficos visuais intuitivos e configurar alertas automatizados.

Motivo da aplicação no cenário da AppFood:

O Grafana atuará como a interface central de observabilidade para a equipe de engenharia. Acoplado ao Prometheus, ele consolidará painéis visuais interativos que permitem monitorar a saúde operacional da aplicação em tempo real. Isso possibilita que a equipe identifique e isole rapidamente anomalias específicas nos módulos críticos — como quedas de performance no microsserviço de *Pagamentos* ou variações no faturamento de *Vendas* —, garantindo uma resposta ágil a incidentes.